KRISH NAIK ACADEMY

PRODUCTION LLM ENGINEERING

RAG, Agents & Fine-Tuning

PRODUCTION LLM ENGINEERING

RAG SYSTEMS

LANGGRAPH & AGENTIC AI

LLMOPS & CI/CD

LLM FINE-TUNING

MULTI-AGENT AI

This course is a focused, advanced journey through production-grade LLM engineering, designed for experienced practitioners who want to build and deploy real-world systems. You will master Transformer architectures, fine-tuning at scale (QLoRA, SFT, DPO), knowledge distillation, and advanced RAG with multimodal and graph retrieval. You will build production multi-agent systems using LangGraph, MCP, A2A protocol, and Bedrock AgentCore. You will also learn LLMOps fundamentals including evaluation harnesses (RAGAS, PromptFoo, Inspect AI), CI/CD with eval-gated pipelines, and deployment to cloud infrastructure. By the end of this *7-month program, you will be capable of building and shipping production-grade LLM systems that reason, retrieve, and collaborate autonomously.

Learning Objectives

- Implement and fine-tune Transformer-based LLMs using QLoRA, SFT, and DPO.
- Apply knowledge distillation to compress large models for edge deployment.
- Build advanced RAG systems integrating ColPali, Neo4j, hybrid retrieval, and reranking.
- Design and deploy multi-agent systems with LangGraph, MCP, and A2A protocol.
- Secure AI systems with guardrails, PII masking, and policy-based access control.
- Evaluate LLM systems using RAGAS, PromptFoo, Inspect AI, and LLM-as-judge.
- Build CI/CD pipelines with eval-gated stages, cost regression, and auto-rollback.
- Deploy models and agents to AWS (SageMaker, ECS, Bedrock AgentCore) with full observability.

Course Information

Prerequisites

Intermediate to advanced Python programming experience (functions, classes, decorators). Solid understanding of machine learning fundamentals (training, evaluation, overfitting). Familiarity with cloud basics (AWS console, IAM, CLI) is helpful but not mandatory. Experience with version control (Git) and command-line interfaces. Prior exposure to LLMs or RAG is beneficial but not required as foundational concepts are covered.

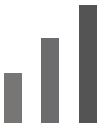
The course is designed to be completed over approximately 7 to 8 months, targeting advanced learners who can move quickly through foundational concepts and spend the majority of time on hands-on implementation and production deployment of the five end-to-end projects

Estimated Time



7 months 8hrs/week*

Required Skill Level

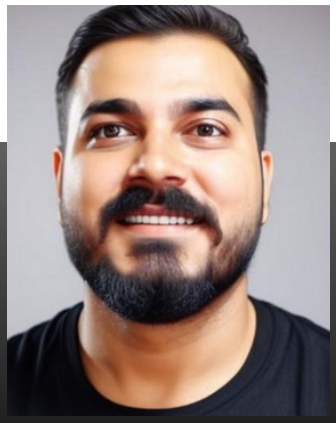


Advance

Krish Naik Academy Team



Sourangshu Pal
Senior Data Scientist
Linkedin



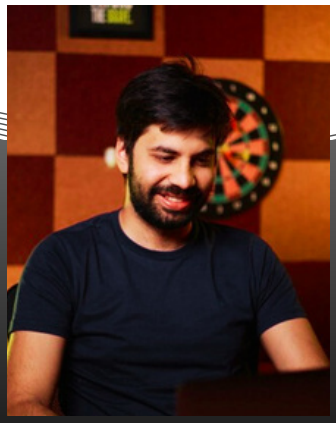
Krish Naik
Chief AI Engineer
Linkedin



Monal Kumar
Data Scientist
Linkedin



Sunny Savita
GenAI Engineer
Linkedin



Mayank Aggrawal
Senior ML Engineer
Linkedin



Darius B.
Head of Product
Linkedin

Module 01

Transformer Architecture & Tokenization Foundations

Understand how transformers process language through embeddings and attention, distinguish between self, masked, and cross-attention, explore encoder-only, decoder-only, and encoder-decoder architectures, and learn the role of tokenization methods like BPE, WordPiece, and SentencePiece in preparing text for LLMs.

Topics	
Transformer Core Mechanics	Embeddings: From Discrete to Continuous Space, The Attention Mechanism, Self Attention, Multihead Attention, Masked Multihead Attention, Positional Encoding, Encoder–Decoder Transformers, Encoder-Only Transformers, Decoder-Only Transformers, Cross-attention
Tokenization Strategies	Taxonomy of Tokenization (Word level, Subword, Character level, Byte level), Byte Pair Encoding (BPE), WordPiece, SentencePiece

Module 02

Fine-Tuning Transformer Architectures in Practice

Implement attention mechanisms from code, and gain hands-on experience fine-tuning encoder-only (DistilBERT), decoder-only (DistilGPT), and encoder-decoder (T5) models on custom datasets for real-world NLP tasks.

Topics	
Attention Implementation	Coding Attention Mechanisms
Fine-Tuning Workflows	Fine-tuning DistilBERT with Custom Data, Fine-tuning DistilGPT with Custom Data, Fine-tuning T5 with Custom Data

Module 03

Inference Optimization, Attention Variants & Scaling Laws

Optimize inference using KV caching and Flash Attention, compare modern attention variants (MQA, GQA, MLA), understand PagedAttention for memory efficiency, apply RoPE for positional encoding, and use scaling laws (Chinchilla) to compute-optimally size models for production.

Topics	
KV Cache & Memory Optimization	The Naive Decoding Problem, KV Cache, KV Cache Memory Math, Flash Attention, PyTorch SDPA — The Unified Attention API
Attention Mechanism Variants	Multi-Head Attention (MHA), Multi-Query Attention (MQA), Grouped-Query Attention (GQA), Multi-Head Latent Attention (MLA), PagedAttention & vLLM
Positional Encoding Schemes	RoPE
Scaling Laws for LLMs	Scaling Laws for Neural Language Models, Training Compute-Optimal Large Language Models, Chinchilla Scaling Laws

Module 04

LLM Fine-Tuning Lifecycle & Pre-Training Foundations

Understand the two-phase LLM lifecycle (pre-training vs post-training), why base models require fine-tuning, deep dive into continued pre-training for domain adaptation, and explore advanced techniques like Multi-Token Prediction.

Topics	
Modern LLM Development Lifecycle	Pre-training vs Post-training — the two-phase view, What pre-training produces (base model), Why base models are not useful out of the box, Training objectives recap: CLM, MLM, Prefix-LM
Pre-Training Deep Dive	Data curation and filtering at scale, Continued Pre-Training (CPT) for domain adaptation, The compute budget problem (links back to Scaling Laws), When CPT is needed vs jumping straight to SFT, Multi-Token Prediction (MTP)

Module 05

Data Preparation & Synthetic Dataset Generation for Fine-Tuning

Master data preparation formats and loss masking, generate high-quality synthetic instruction and preference datasets using self-instruct and LLM-as-Judge techniques, use modern tooling to avoid model collapse, and build complete domain-specific SFT datasets.

Topics	
Data Preparation for Fine-Tuning	Dataset formats (instruction pairs, chat format), Chat templates: ChatML, LLaMA-3, Mistral, Loss masking — why it matters, what breaks without it, Deduplication and filtering pipelines
Synthetic Dataset Generation	Why Data Is the Leverage Point, The Taxonomy of Synthetic Data, Instruction Dataset Generation (Self-Instruct, Alpaca: Scaling Self-Instruct), Preference Dataset Generation, LLM-as-Judge & LLM-as-Judge Scoring, Tools: distilabel, DataDreamer, Argilla, Risks: Model Collapse and Data Poisoning, End-to-End Pipeline: Building a Domain SFT Dataset

KRISH NAIK ACADEMY

Module 06

Supervised Fine-Tuning, Parameter-Efficient Methods & Preference Alignment

Apply LoRA, QLoRA, and advanced PEFT variants for memory-efficient fine-tuning; perform SFT for instruction following, chat, and chain-of-thought; and align models to human preferences using RLHF and DPO without complex reward modeling.

Topics	
Parameter-Efficient Fine-Tuning (PEFT)	The intrinsic dimensionality insight, LoRA — math, rank, alpha, target modules, QLoRA — 4-bit NF4, double quantization, paged optimizers, DoRA — magnitude + direction decomposition, AdaLoRA — adaptive rank allocation, LoRA+ — separate learning rates for A and B
Supervised Fine-Tuning (SFT)	SFT as Stage 1 of every post-training pipeline, Instruction tuning (FLAN, Alpaca, OpenHermes), Chat / conversational fine-tuning, Chain-of-Thought fine-tuning, Domain-specific fine-tuning best practices
Preference Alignment	Why SFT alone is not enough, RLHF with PPO — reward model, critic, KL penalty, DPO — direct optimization, no reward model

Module 07

Evaluation, Quantization, Deployment & Fine-Tuning Tooling

Evaluate fine-tuned models using benchmarks and LLM-as-judge, quantize models (GPTQ, AWQ, GGUF) for efficient serving, deploy with vLLM, SGLang, or llama.cpp, serve multiple LoRA adapters from one base model, and leverage production-grade fine-tuning frameworks like TRL, Unsloth, and Axolotl.

Topics	
Evaluation	Why evaluation is part of the fine-tuning workflow, Benchmark types: knowledge, reasoning, instruction-following, LLM-as-judge: MT-Bench, Chatbot Arena, Domain-specific evaluation design
Quantization & Deployment Prep	GPTQ, AWQ, BNB NF4, FP8, Merging LoRA adapters before serving, Inference frameworks: vLLM, SGLang, Serving multiple LoRA adapters from one base model, GGUF format, llama.cpp, Speculative Decoding & Inference
Tooling & Frameworks	HuggingFace TRL — SFT, DPO, GRPO, ORPO, Unsloth — consumer GPU optimization, Axolotl — full YAML configurability, LLaMA-Factory — no-code web UI, Together AI / AWS SageMaker — managed fine-tuning

Module 08

Mixture of Experts (MoE) Architecture for Scaling LLMs

Understand why dense models hit scaling limits, how MoE activates only relevant experts per token, master load balancing to avoid expert collapse, compare training and inference trade-offs, explore variants like sparse MoE and soft MoE, and decide when MoE outperforms dense architectures in production.

Topics	
Mixture of Experts	The Dense Model Scaling Problem, The MoE Idea, Architecture Deep Dive, Load Balancing, Training MoE Models, Inference with MoE, MoE Variants, Production MoE Models, MoE vs Dense: When to Use Each

Module 09

Reasoning Models, Chain-of-Thought & RL-Only Training

Define reasoning models and their distinction from standard LLMs, implement chain-of-thought prompting and training, explore DeepSeek's approach to RL-only reasoning without supervised fine-tuning, and apply distillation techniques to transfer reasoning capabilities to smaller, efficient models.

Topics	
Reasoning Models	What Is a Reasoning Model?, Chain-of-Thought: The Foundation, The Training Recipe, DeepSeek-R1-Zero: Skipping SFT Entirely, Distillation: Reasoning Without RL

Module 10

Small Language Models & Knowledge Distillation

Understand why Small Language Models are critical for production (cost, latency, privacy), master knowledge distillation with soft labels, temperature scaling, and KL divergence, explore complementary techniques like pruning and quantization, and build end-to-end distillation pipelines to compress powerful LLMs into efficient SLMs without catastrophic performance loss.

Topics	
Small Language Models	What Is a Small Language Model?, Why SLMs Matter, The SLM Design Philosophy, Key Techniques for Building SLMs (Knowledge Distillation, Pruning, Quantization, Liquid Foundation Models), Training SLMs from Scratch, Fine-Tuning SLMs, Reasoning in SLMs, SLM vs LLM: Choosing the Right Scale
Knowledge Distillation	The Student-Teacher Paradigm, The Core Problem: What Is Knowledge?, Hard Labels vs Soft Labels, Temperature Scaling, The KL Divergence Loss, Attention Transfer, Building a Distillation Pipeline (Step 1: Teacher Data Generation, Step 2: Student Architecture Design, Step 3: Training the Student)

Module 11

Vision Foundations: From CNNs to Vision Transformers

Transition from CNN to Vision Transformer understanding, implement patch embedding, CLS token, and 2D positional encoding, visualize attention maps to interpret ViT, compare inductive biases of CNNs vs ViTs, and master pre-trained vision encoders (CLIP, SigLIP, DINOv2) for downstream tasks.

Topics	
From CNN to ViT Architecture	From CNN to Vision Transformer — The Conceptual Bridge, ViT Architecture Deep Dive, Image as a Sequence of Patches, Patch Embedding, The CLS Token, Positional Encoding for 2D Inputs, Transformer Encoder on Visual Tokens
Attention Visualization & CNN vs ViT Tradeoffs	Attention Maps — What ViT Sees, CNN vs ViT — Inductive Bias and the Core Tradeoff
Pre-Trained Vision Encoders	Pre-Trained Vision Encoders, CLIP — Contrastive Language-Image Pre-training, SigLIP — Sigmoid Loss Variant, DINO / DINOv2 — Self-Supervised Vision

Module 12

Visual Language Models (VLMs): Connecting Vision and Text

Define Visual Language Models and their multimodal capabilities, dissect the three-component VLM architecture (visual encoder + aligner/projector + LLM backbone), understand how the projector maps visual tokens into LLM embedding space, and reason through training strategies for vision-language alignment.

Topics	
Visual Language Models	What Is a Visual Language Model?, The VLM Architecture, The Visual Encoder, The Aligner / Projector, The Language Model Backbone, How Vision and Language Are Connected

Module 13

Speech-to-Text Models & Fine-Tuning Whisper

Understand the landscape of Speech AI and Speech-to-Text systems, explore Whisper's architecture and API, build production-ready STT pipelines, prepare custom speech datasets, and fine-tune Whisper on domain-specific audio data for improved transcription accuracy.

Topics	
Speech AI & STT Foundations	Introduction to Speech AI, Speech-to-Text Foundations, Whisper Architecture
Whisper API & STT Pipelines	Speech-to-Text with Whisper API, Building STT Pipelines
Fine-Tuning Whisper	Dataset Preparation for Fine-Tuning, Fine-Tuning Whisper on Custom Data

Module 14

Embedding Models: Taxonomy, Matryoshka & Fine-Tuning

Navigate the embedding model taxonomy from dense to binary, understand trade-offs between semantic richness and compression, implement Matryoshka Representation Learning (MRL) for flexible embedding dimensions at query time, produce MRL embeddings in production, and apply fine-tuning strategies to optimize embeddings for domain-specific retrieval.

Topics	
Embedding Taxonomy & Types	Advanced Embedding Taxonomy, Dense Embeddings — The Baseline, Sparse Embeddings — Lexical Signals Preserved, Quantized Embeddings — Float32 → Int8 / Uint8, Binary Embeddings — Maximum Compression, Multi-Vector Embeddings — One Document, Many Vectors
Matryoshka Embeddings (MRL)	Matryoshka Embeddings — Flexible Dimensions at Query Time, How MRL Training Works, Using MRL Embeddings in Production
Embedding Fine-Tuning	Embedding Fine-Tuning Strategies, Embedding Fine-Tuning

Module 15

LangChain for RAG: Ecosystem, Patterns & Production

Master LangChain as the orchestration framework for production RAG systems, understand LCEL for composable chains, implement memory, tool calling, and structured output, trace and evaluate with LangSmith, and deploy secure, scalable RAG APIs.

Topics	
Architecture, Integrations & LCEL	Architecture & Ecosystem 2026, LLM Integrations & Model Abstraction, LCEL Deep Dive, Integrations Ecosystem 2026
Prompting, Output Parsing & Tool Calling	Prompt Engineering, Structured Output & Output Parsers, Tool Calling & Function Calling
Memory, Retrieval & RAG	Memory & Conversation History, Document Loaders & Text Splitters, Embeddings & Vector Stores, Retrieval & Advanced RAG, Agentic RAG & Advanced Pipelines
Agents, Multimodal & Text-to-SQL	Agents — Legacy & Modern, Multimodal — Images, Audio, Documents, Text-to-SQL & Structured Data
Production, Observability & Security	Callbacks, Tracing & LangSmith, Production Patterns & Best Practices, Building & Deploying LangChain APIs, Security, Safety & Responsible AI, Evaluation, Testing & LangSmith Evals

Module 16

Foundations of RAG: Vanilla, Embeddings, Chunking & Retrieval

Build baseline RAG systems from scratch, choose optimal embedding models and chunking strategies, and implement hybrid retrieval using BM25, SPLADE, and ColBERT-style multi-vector retrieval.

Topics	
Foundations of RAG	Vanilla RAG, Embedding models, Chunking Strategies, BM25, SPLADE & Multi Vector COLBERT

Module 17

Advanced RAG: Query Transformations, Rerankers & Adaptive Retrieval

Transform queries for better retrieval, apply rerankers to improve precision, implement self-correcting and adaptive RAG patterns, evaluate RAG pipelines systematically, and build agentic RAG where the LLM decides when and how to retrieve.

Topics	
Advanced RAG Systems	Hybrid RAG, Meta Hybrid RAG, Query Transformations, RAG Evaluations, Rerankers, Self RAG, Corrective RAG, Adaptive RAG, Contextual Retrieval, Agentic RAG

Module 18

Vector Quantization, Multimodal RAG & Emerging Patterns

Scale vector search with product, scalar, and binary quantization; build multimodal RAG pipelines with vision-language models and ColPali; implement Graph RAG for knowledge graph retrieval; optimize with semantic caching; and secure RAG systems against prompt injection with guardrails and PII masking.

Topics	
Vector Quantization for Scaling	Scalar Quantization, Binary Quantization, Product Quantization
Multimodal RAG	ColPali Paradigm, Single Stage & Dual Stage Data Parsing, Layout Detection, OCR Paradigm, Structure Layout Chunking, VL Embeddings, VL Rerankers, Multimodal LLMs
Graph RAG	Building Graph RAG with graph databases like Neo4j
Vectorless RAG	PageIndex
Caching in RAG	Caching, Semantic Caching
Securing RAG	PI Masking, Guardrails, Input and Output Guardrails, Prompt Injection Defense

Module 19

Agent Foundations: Structured Outputs, Function Calling & MCP

Validate and structure agent outputs with Pydantic, implement cross-provider function calling (OpenAI, Anthropic, Google), build tool executor loops, and standardize tool integration using MCP architecture (hosts, clients, servers, stdio/SSE transport).

Topics	
Pydantic — Data Validation & Structured Outputs	Installation, Models, Fields, Validators, JSON parsing, type coercion, Dataclasses, nested models, Validation decorators, custom validators, settings management
Function Calling Fundamentals	Why Function Calling Exists — The LLM-to-Tool Communication Problem, Function Schema Design — JSON Schema, Descriptions, and Parameter Typing, The Function Calling Request-Response Lifecycle
Parallel Calls, Constrained Choice & Structured Output (Function Calling)	Parallel Function Calling — Multiple Tool Calls in a Single Response, Forced and Constrained Tool Choice — required, none, and Specific Tool Selection, Structured Output vs Function Calling — Differences and When to Use Each

Topics	
Tool Executor Loop & Cross-Provider Support (Function Calling)	Building a Tool Executor Loop from Scratch, Function Calling Across Providers — OpenAI, Anthropic, Google, and the Universal Contract
MCP (Model Context Protocol)	Why MCP Exists — The Tool Fragmentation Problem, MCP Architecture — Hosts, Clients, and Servers, MCP Primitives — Tools, Resources, and Prompts, MCP Transport Layer — stdio and SSE, Building an MCP Server from Scratch, Building an MCP Client and Connecting to Servers, MCP Security — Authentication, Authorization, and Trust Boundaries, MCP in Production — Multi-Server Orchestration with LangGraph and Claude

Module 20

LangGraph: Stateful, Multi-Agent & Human-in-the-Loop Workflows

Build stateful agent workflows using LangGraph's graph-based architecture, implement ReACT pattern for tool-calling agent, add human-in-the-loop checkpoints, manage memory and persistence across sessions, orchestrate multi-agent systems, and deploy agents using the LangGraph Platform.

Topics	
LangGraph	Foundations & Setup, Core Graph Concepts, State Management, Nodes, Edges & Routing, Tool-Calling & ReACT, Human-in-the-Loop, Memory & Persistence, Multi-Agent Systems, Streaming & Observability, LangGraph Platform

Module 21

Production Agents: Observability, A2A Protocol & Amazon Bedrock AgentCore

Implement production observability for agents with LangSmith and Logfire, enable agent-to-agent communication using the A2A protocol (Agent Cards, Task Lifecycle, HTTP/SSE transport), deploy secure, isolated agents with Amazon Bedrock AgentCore, manage memory and identity, enforce guardrails with Cedar policies, and orchestrate multi-agent systems at scale.

Topics	
Agent Observability Tools	LangSmith, Logfire
A2A (Agent-to-Agent) Protocol	Why A2A Exists — The Multi-Agent Interoperability Problem, A2A Architecture — Agents, Agent Cards, and the Task Lifecycle, Agent Cards — Capability Discovery and the Well-Known Endpoint, A2A Task Model — Submitted, Working, Completed, Failed, and Streaming States, A2A Transport — HTTP, SSE, and JSON-RPC Message Structure, Building an A2A-Compliant Agent Server, Building an A2A Client — Discovering and Delegating to Remote Agents, A2A vs MCP — When to Use Which and How They Complement Each Other

Topics	
AgentCore Overview & Runtime (Amazon Bedrock AgentCore)	Introduction to Amazon Bedrock AgentCore & the Agentic AI Stack, AgentCore Runtime — MicroVM Isolation, Session Management & Execution Windows
Building & Integrating Agents (Amazon Bedrock AgentCore)	Building Your First Agent with Strands Agents Framework, Integrating LangGraph with AgentCore (Framework-Agnostic Deployment)
Memory, Gateway & Identity (Amazon Bedrock AgentCore)	AgentCore Memory — Short-Term, Long-Term & Cross-Session Persistence, AgentCore Gateway — Exposing APIs, Lambda Functions & MCP Servers as Agent Tools, AgentCore Identity — IAM, OAuth, User-Delegated Access & Secrets Management
Browser, Code Execution & Deployment (Amazon Bedrock AgentCore)	AgentCore Browser & Code Interpreter — Web Automation & Secure Code Execution, Deploying Agents to Production — AgentCore CLI, CodeBuild, ECR & ARM64 Runtime
Multi-Agent, Observability & Guardrails (Amazon Bedrock AgentCore)	Multi-Agent Orchestration — Supervisor Patterns, Agent-to-Agent (A2A) Protocol, AgentCore Observability — CloudWatch, X-Ray, OTEL & Third-Party Monitoring (Langfuse), AgentCore Policy & Guardrails — Cedar Policies, Real-Time Tool Call Interception & Compliance

Module 22

Prompt Engineering: Structure, Techniques & Refinement

Deconstruct prompt anatomy (instruction, context, input, format), apply few-shot, chain-of-thought, and step-back prompting, design effective system prompts and role assignments, enforce structured output (JSON, XML), test prompt robustness, and implement chaining, meta-prompting, and self-refinement loops.

Topics	
Prompt Anatomy & Basic Techniques	Anatomy of a Prompt (instruction, context, input, output format), Zero-Shot, One-Shot, and Few-Shot Prompting, System Prompt Design and Role Assignment
Advanced Reasoning & Structured Output	Chain-of-Thought and Step-Back Prompting, Output Formatting and Structured Generation (JSON mode, XML tags)
Prompt Testing, Chaining & Self-Refinement	Prompt Sensitivity, Fragility, and Robustness Testing, Prompt Chaining and Decomposition Strategies, Meta-Prompting and Self-Refinement Loops

Module 23

Context Engineering: Memory, Pruning & Retrieval-Augmented Context

Understand context window anatomy (tokens, ordering, recency bias), treat RAG as context construction, choose chunking strategies for context quality, implement multiple memory architectures (short-term, episodic, semantic, procedural), compress and prune contexts with LLMLingua and RECOMP, and manage multi-turn conversation state.

Topics	
Context Window Anatomy & RAG as Context Construction	Anatomy of a Context Window (tokens, ordering, recency bias), Retrieval-Augmented Generation as Context Construction, Chunking Strategies and Their Impact on Context Quality
Memory Architectures & Structured Context	Memory Architectures (short-term, episodic, semantic, procedural), Tool Results and Structured Data as Context
Context Compression, Pruning & Multi-Turn State	Context Compression and Pruning (LLMLingua, RECOMP), Multi-Turn Conversation State and Context Accumulation

Module 24

Harness Engineering: Evaluation, Benchmarking & Agent CI/CD

Replace ad-hoc testing with structured evaluation harnesses, benchmark LLMs using lm-evaluation-harness (MMLU, GSM8K, TruthfulQA), build agent-native evals with Inspect AI, implement LLM-as-Judge pipelines, secure tool execution with sandboxes (E2B, Modal, Docker), test prompt regression with PromptFoo, and build agent CI/CD pipelines with eval gating and cost regression guards on PRs.

Topics	
Evaluation Fundamentals & LLM Benchmarking	Evaluation Harness Fundamentals (what evals are, why ad-hoc testing fails), LLM Benchmarking with lm-evaluation-harness (MMLU, GSM8K, TruthfulQA)
Agent-Native Evaluation & LLM-as-Judge	Agent-Native Eval with Inspect AI (Task, Solver, Scorer abstractions), LLM-as-Judge Pipelines (model-graded scoring, calibration, bias)
Execution Sandboxes, State Management & Agent Harness Patterns	Tool Execution Sandboxes (E2B, Modal, Docker — secure code execution), Agent State Management & Checkpointing (LangGraph persistence, interrupts), ReAct Loop Harness Patterns (iteration guards, fallback handlers, token budgets), Multi-Agent Execution Harnesses (supervisor-worker contracts)

Topics	
Prompt Regression, Trace-Based Testing & Agent CI/CD	Prompt Regression & Snapshot Testing (PromptFoo, golden datasets, versioning), Trace-Based Testing & Observability (Opik/LangSmith assertions, flaky test detection), Agent CI/CD Pipelines (GitHub Actions, eval gating on PRs, cost regression guards)

Project 01

MedScript AI: Domain-Specific LLM Fine-Tuning for Healthcare

Execute a complete medical LLM fine-tuning pipeline using QLoRA SFT and DPO preference alignment, generate synthetic instruction data, evaluate model quality using automated metrics and LLM-as-judge, and deploy a multi-adapter vLLM server on AWS with hot-swappable LoRA adapters per request.

Topics	
Project Overview & Learning Outcomes	Build a domain-specific medical language model by running a full post-training pipeline on open-source clinical data, Cover synthetic data generation, QLoRA SFT, DPO preference alignment, and multi-adapter production serving with vLLM
Dataset & Training Configuration	ChatDoctor-HealthCareMagic (patient Q&A pairs for SFT), PubMedQA (clinical reasoning QA pairs), MedAlpaca / Medical Meadow Wikidoc, Base model: Llama-3.1-8B-Instruct, Stage 1 — QLoRA SFT with rank, alpha, and 4-bit NF4 configuration on mid-tier GPU, Stage 2 — DPO with beta and sigmoid loss over one epoch

Topics	
Tech Stack & Implementation Stages	distilabel for synthetic instruction data generation (Self-Instruct style), HuggingFace datasets for deduplication and chat format with loss masking, transformers + TRL (SFTTrainer, DPOTrainer), Unsloth for VRAM reduction, PEFT for LoRA adapter management, bitsandbytes for 4-bit NF4 quantization
Evaluation Strategy	ROUGE-L and BERTScore for text similarity, LLM-as-judge (GPT-4o or Claude) on medical accuracy, safety, and tone, Compare base model vs SFT vs SFT+DPO performance
Production Deployment	vLLM serving with one base model and multiple LoRA adapters hot-swapped per request, FastAPI REST API with adapter routing by endpoint, Containerisation with Docker, Deployment to AWS SageMaker Endpoint, LangSmith for tracing
Project Deliverables	Cleaned SFT dataset and DPO preference pairs pushed to HuggingFace Hub, Trained QLoRA adapter (Stage 1) and DPO-aligned adapter (Stage 2), Evaluation report comparing all model variants, Live FastAPI endpoint with multi-adapter vLLM server containerised on AWS

Project 02

EdgeReason: Knowledge Distillation from Large to Small Model for CPU Inference

Implement knowledge distillation from scratch using KL divergence and attention transfer loss, run ablation studies to measure compression-quality tradeoffs, quantise models to GGUF format, and deploy CPU-based inference with llama.cpp at production cost levels.

Topics	
Project Overview & Learning Outcomes	Compress a larger teacher model into a smaller student model using KL divergence and attention transfer loss implemented from scratch, Quantise to GGUF format for CPU inference, Close majority of the quality gap at a fraction of serving cost
Dataset & Training Configuration	GSM8K maths problems (primary distillation and evaluation), MATH problems (extended reasoning), MetaMath augmented problems (optional), Teacher and student model selection, Temperature parameter for soft label smoothing, Combined loss function with weights for KL divergence, attention transfer, and cross-entropy, Training on mid-tier GPU over several hours

Topics	
Tech Stack & Implementation Stages	transformers + PyTorch for logit extraction from teacher and custom training loop, wandb for loss curve tracking, KL divergence and Attention Transfer implemented from scratch, Ablation study comparing different loss configurations against teacher baseline
Quantisation & CPU Inference	llama.cpp with GGUF conversion support, Multiple quantisation levels (high precision to aggressive compression), llama-server for OpenAI-compatible CPU inference API, Benchmark: tokens per second and first-token latency comparison against teacher
Project Deliverables	Soft labels (teacher logits at temperature) saved for all training problems, Trained student checkpoint with combined distillation loss, Ablation comparison table across loss configurations, GGUF quantised model file, llama-server running on CPU, Quantisation benchmark report

Project 03

LexisGraph: Enterprise Legal RAG with ColPali, Neo4j & Hybrid Retrieval

Build a production legal RAG system combining ColPali (no OCR), Neo4j knowledge graphs, BM25, dense retrieval, RRF fusion, and cross-encoders, implement adaptive query routing, secure PII and enforce guardrails, and use RAGAS evaluation with gated metrics.

Topics	
Project Overview & Learning Outcomes	Enterprise legal document intelligence combining ColPali multimodal indexing (no OCR), Neo4j knowledge graph, hybrid BM25 and dense retrieval, Reciprocal Rank Fusion, cross-encoder reranking, and RAGAS-gated evaluation harness
Dataset & Document Parsing (No OCR)	CUAD commercial contracts with clause categories (primary dataset), Public court documents (optional), Multi-jurisdiction legal text (optional), pdf2image for PDF to image conversion, ColPali VLM for producing patch vectors per page, Native handling of scanned PDFs, tables, charts, and signatures
Production Deployment	FastAPI + LangChain LCEL + Docker for serving layer

Topics	
Retrieval Stack & Vector Database	Qdrant for multi-vector index with MaxSim (late interaction / ColBERT-style) scoring, Elasticsearch for BM25 sparse retrieval, Neo4j graph database with nodes for contracts, parties, clauses, obligations, dates, and amounts, Entity extraction using LLM + Pydantic
Fusion, Reranking & Routing	Reciprocal Rank Fusion across all retrievers, Cross-encoder reranker for refining top results, Adaptive query router: visual queries to ColPali, exact clause to BM25, relational/multi-hop to Neo4j, complex queries to all retrievers with fusion
Security & RAGAS Evaluation	presidio for PII masking before retrieval, NeMo Guardrails for input (injection) and output (hallucination) guardrails, RAGAS evaluation metrics: Faithfulness, Answer Relevancy, Context Precision, Context Recall, Gate threshold on faithfulness using golden QA pairs
Project Deliverables	Full corpus indexed in Qdrant with multi-vector ColPali, Neo4j graph populated with entities and relationships from all documents, Hybrid retrieval with BM25, ColPali, Neo4j, RRF fusion, and reranker operational, Adaptive query router correctly classifying across all retriever types, PII masking and guardrails integrated into pipeline, RAGAS report on golden test cases meeting faithfulness gate

Project 04

AutoOps: Production DevOps Multi-Agent System with Human-in-the-Loop & A2A

Build a production multi-agent system with LangGraph supervisor and conditional routing, implement MCP servers for tool exposure, enable A2A peer delegation between agents, integrate HITL with checkpoint persistence, deploy to cloud agent runtime with microVM isolation, and enforce policy-based access control for agent permissions.

Topics	
Project Overview & Learning Outcomes	Production DevOps assistant with LangGraph supervisor delegating to specialised sub-agents (GitHub, CloudWatch, Code Review, Monitoring), All agents expose MCP servers, Human-in-the-loop gates all write operations, Deployed to cloud agent runtime with policy-based access control
Agent Architecture & MCP/A2A Protocols	LangGraph supervisor-worker graph with conditional routing and shared state, FastMCP (Python) for MCP servers across all agents with stdio (development) and SSE (production) transport, GitHub agent wrapping CLI as MCP tools, CloudWatch agent wrapping AWS SDK as MCP tools, Code Review agent with static analysis and security scanning, Monitoring agent for service health and deployment history, A2A peer delegation between agents, A2A discovery via standard endpoints with JSON-RPC transport

Topics	
Human-in-the-Loop & State Management	HITL interrupts before write operations, SQLite state serialisation on interrupt with exact checkpoint resume after approval
Cloud Agent Runtime Deployment	MicroVM isolation per session, Gateway registering all MCP servers as tools, Memory: short-term session state and long-term cross-session semantic memory, Identity: IAM roles per agent with OAuth and secrets management
Policy-Based Access Control & Observability	Policy language for agent permissions (write operations require human approval, read-only agents restricted from destructive actions), Observability: Tracing platforms, metrics, and structured logging across all agents
Demo Scenario & Project Deliverables	Demo: triage production incidents leading to automated GitHub issue creation with code review and human approval in under a minute, Deliverables: LangGraph supervisor with sub-agents and shared state, All MCP servers implemented and registered with gateway, HITL interrupt and resume working for all write operations, A2A peer delegation working end-to-end, All agents deployed to cloud agent runtime with policies and observability live

Project 05

ShipLLM: LLMOps CI/CD with Eval-Gated Deployment for All Projects

Build an LLMOps CI/CD pipeline with eval-gated stages, enforce regression, cost, and performance gates, implement blue/green deployments with auto-rollback, containerise all services with multi-stage builds, and maintain a golden test suite across diverse model and agent types.

Topics	
Project Overview & Learning Outcomes	Wrap all prior fine-tuning, distillation, RAG, and multi-agent projects under a production LLMOps shell, Every code or prompt change must pass eval-gated CI stages before deployment, All model and agent services have live cloud endpoints gated by same pipeline with auto-rollback
CI/CD Pipeline Stages (Hard Eval Gates)	Stage 1 — Linting and unit tests, Stage 2 — Prompt regression testing against golden baseline, Stage 3 — Evaluation harness score gate per service, Stage 4 — Cost and performance regression guard, Gate — Pull request blocked if any eval stage fails with score delta posted as PR comment, Stage 5 (on merge) — Container build and push to registry with rolling deployment, Stage 6 — Smoke tests on staging with golden cases

Topics	
Eval Gate Threshold	Faithfulness metrics meet minimum bar, Answer relevancy meets minimum bar, Zero prompt snapshot regressions allowed, Tokens per request within tolerance of baseline, Cost per requests within tolerance of baseline, Latency percentiles within tolerance of baseline
Tech Stack & Cloud Infrastructure	CI/CD platform for pipeline orchestration, Prompt regression testing framework with golden snapshot store and rubric-based assertions, Agent-native evaluation framework for multi-step agent traces, Standardised LLM benchmarking harness for base model evaluation, RAG evaluation framework for retrieval quality metrics, LLM-as-judge with calibrated prompts per domain, Multi-stage container builds for all services, Container registry with per-service repositories, Container orchestration service for stateless API services, Model serving platform for fine-tuned model endpoints, Blue/green deployment strategy with gradual traffic shifting, Cloud monitoring with auto-rollback on error rate breach

Topics	
Deployment & Observability	Blue/green deployment with gradual traffic shifting and observation window, Cloud monitoring alarms for auto-rollback on error rate threshold breach, Tracing platform for per-call traces, token cost, and prompt versions, Self-hosted or managed observability for LLM metrics, Custom dashboard for LLM metrics, Golden test cases committed to repository as regression baseline
Project Deliverables	Golden test cases (per-project splits) committed to repository, Regression testing configuration per project with rubric, cost, and latency assertions, Full CI pipeline YAML with all stages, Multi-stage container files for all project services meeting size constraints, All services live on cloud with appropriate serving infrastructure, Monitoring dashboard with LLM metrics and auto-rollback alarm configured, Live demo showing prompt change triggering eval gate and deployment with rollback demonstrated